# KG 구축 how 보완
# Relation 잇기 · 저장 · 검색·답변

따른 문헌의 원리와 실제 수행 과정

2026.11 · 문경민

## 1. KG 구축 단계와 방법론 구체화 현황

- KG 구축은 두 단계로 한다. 먼저 온톨로지의 Class 와 Data Property 에 맞춰 자료에서 실제값으로 Entity 와 그 Data Property 를 만들고, 다음에 만든 Entity 들을 온톨로지의 Relation 에 맞춰 이어 KG 를 완성한다.
- 10/30 자료에서 앞 단계를 정리했고, 그 결과 Entity 264 개와 Data Property 값 616 개가 만들어졌다. 이번 자료는 Entity 를 Relation 으로 잇는 과정(2 장), 완성한 KG 를 Neo4j 에 넣는 저장(3 장), KG 로 질문에 답하는 검색·답변(4 장)을 같은 방식(따른 원리 → 과정 → 결과)으로 정리했다.
- 온톨로지의 CQ 와 Relation 은 부록 B, 10/30 자료까지의 결과는 부록 C, 각 원리의 자세한 설명은 부록 D 에 있다.

### 표 1. 단계별 하는 일과 방법론 정리 여부

| 단계 | 하는 일 | 방법 | 방법론 정리 |
|---|---|---|---|
| 자료 준비 | 소스 구분과 전처리 | — | 정리함 (10/30) |
| Class·Data Property 채우기 | Entity 와 속성값을 채움 | 구조화 자료 매핑(정형 표), 비정형 텍스트 추출(비고, 별표 3) | 정리함 (10/30) |
| Relation 잇기 | 만든 Entity 를 Relation 마다 실제 시작·끝 Entity 로 이음 | 키로 이을 끝 Entity 를 찾음(댐 표기 맞추기 포함) | 이번 (2 장) |
| 저장 | 완성한 KG 를 Neo4j 에 넣음 | CSV → Cypher → Neo4j | 이번 (3 장) |
| 검색·답변 | 질문에 맞는 근거를 KG 에서 찾아 LLM 이 답함 | 의도 추출 → Cypher 템플릿 조회 → 근거 묶음 → 답변 | 이번 (4 장) |
| 검증 | 만든 KG 가 맞는지 확인 | — | 선행연구 조사 후 |

## 2. Relation 잇기

### 2.1 Relation 잇기란

- 그림 1 의 왼쪽은 온톨로지의 Relation 정의 "Approval —concernsDam→ Dam"이다. 어느 승인이 어느 댐에 관한 것인지는 들어 있지 않다.
- 오른쪽은 KG 의 일부이다. 정의에 맞는 조합은 승인 80 개 × 댐 4 개 = 320 가지이지만(회색 점선), 실제 Relation 은 80 개뿐이다. 승인 504 행의 관측소코드 1003110 이 충주댐의 댐 코드와 같으므로 APR:504 를 DAM:1003110 과 잇는다(실선).
- Entity 와 Data Property 값만 있는 KG 에는 선이 없어서 "충주댐의 승인"을 조회해도 아무것도 나오지 않는다. 이번 단계에서 Entity 를 Relation 으로 이어 KG 를 완성한다.

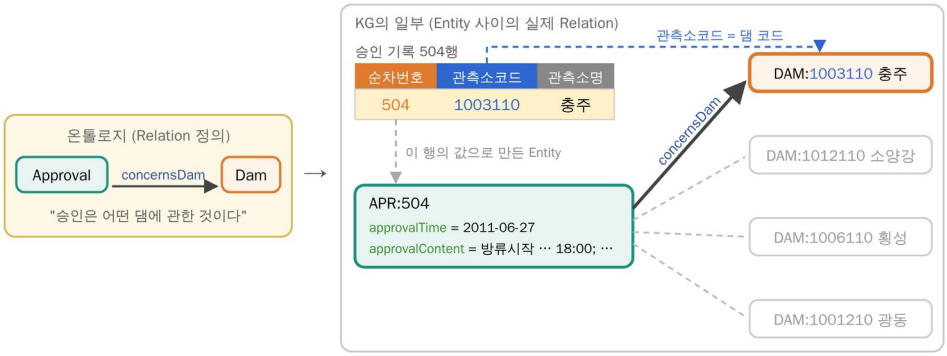



그림 1. 온톨로지의 Relation 정의에 맞는 조합(회색 점선) 가운데, 승인 기록의 관측소코드로 실제로 이을 댐(실선)을 찾는다

## 2.2 따른 원리와 진행 방식

- 따른 원리(Purohit 외, 2024; 부록 D.1): 기준 식별자를 하나 정하고, 자료마다 다르게 적힌 같은 대상을 그 식별자에 대응시킨다. 우리는 K-water 댐 코드를 기준 식별자로 삼았고, 대상이 댐 4 개라서 이번에는 코드 일치와 대응 표로 맞췄다.
- 진행 방식: 댐 표기를 댐 코드에 맞춘 뒤, Relation 마다 시작 Entity 를 하나씩 읽고 키로 이을 끝 Entity 를 찾아 Relation 한 줄을 쓴다. 키는 두 Entity 가 같은 대상에 관한 것임을 알려 주는 값(승인 순차번호, 댐 코드, 댐 이름)이다.

## 2.3 댐 표기 맞추기 (개체 해소)

- 그림 2 의 왼쪽은 충주댐이 자료마다 적힌 모습이다. ① 승인 기록의 관측소코드는 K-water 댐 코드와 같아서 그대로 맞춘다(자료 준비 때 4 개 댐 모두 같음을 확인).
- ② K-water 자료의 DAM_CD 는 기준 식별자 자체이다.
- ③ 규정 별표 3 에는 이름 "충 주 댐"뿐이라, 공백을 빼고 읽은 "충주댐"을 대응 표(4 줄)에서 찾아 1003110 을 얻는다. 광동댐은 별표 3 에 없다.
- 결과: 세 표기가 모두 하나의 Entity DAM:1003110 을 가리킨다.

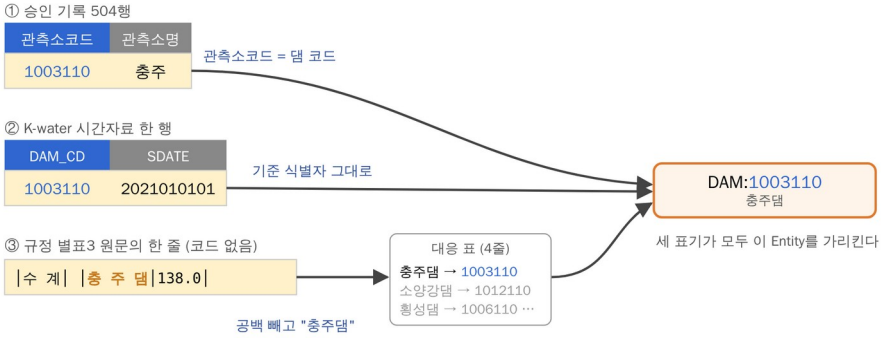



그림 2. 자료마다 다른 충주댐 표기 세 가지를 기준 식별자 DAM:1003110 에 맞춘다

## 2.4 승인 기록 한 행으로 잇는 Relation

- 그림 3 의 위쪽은 승인 기록 원본(CSV)의 504 행이다. 범례처럼 회색 점선은 앞 단계에서 Entity 를 만든 길, 파란 점선은 키 비교, 검은 실선은 이번에 이은 Relation 이다.
- 앞 단계에서 이 행으로 만든 APR:504, OPR:504:1, EVI:AP:504 의 ID 에는 모두 순차번호 504 가 들어 있다. 그래서 ① APR:504 → OPR:504:1(authorizes), ② APR:504 → EVI:AP:504 와 OPR:504:1 → EVI:AP:504(supportedBy)는 순차번호가 같은 것끼리 잇는다.
  ③ APR:504 → DAM:1003110(concernsDam)과 OPR:504:1 → DAM:1003110(performedOnDam)은 이 행의 관측소코드 1003110 과 같은 댐 코드를 가진 DAM:1003110 을 끝 Entity 로 찾는다.
- 결과: 승인 80 행에 같은 방법을 적용해 supportedBy 148 개(승인 80, 운영행위 68), authorizes 68 개, concernsDam 80 개, performedOnDam 68 개를 잇는다. 변경 승인 17 개는 운영행위가 없어 authorizes 가 없다.

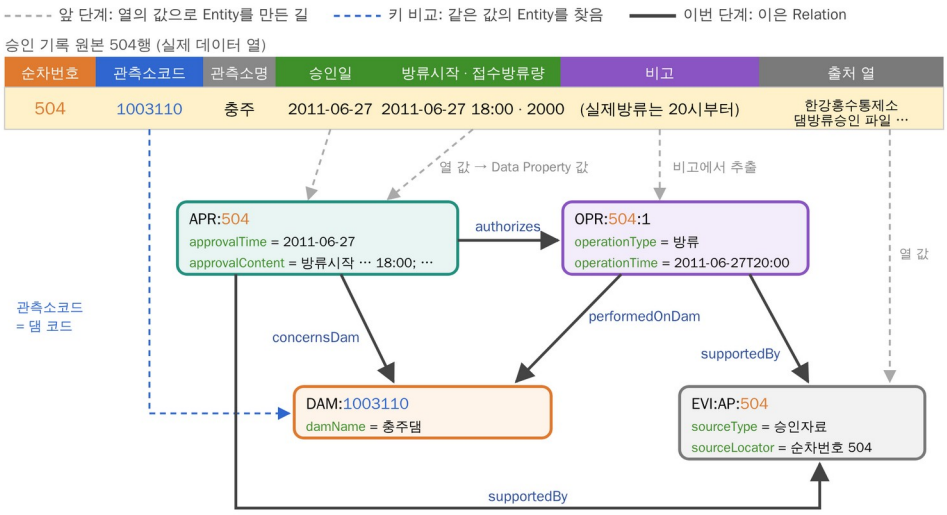



그림 3. 승인 504 행 하나에서 잇는 Relation — 순차번호 504 와 관측소코드 1003110 으로 이을 Entity 를 찾는다

## 2.5 댐 코드로 잇는 Relation

- 그림 4 의 위쪽은 K-water 시간자료의 충주댐 한 행이다. 앞 단계에서 이 행으로 만든 HMS 6 개와 EVI:KW:1003110, 일자료로 만든 DAM:1003110 의 ID 에 모두 댐 코드 1003110 이 들어 있다.
- 그래서 ① DAM:1003110 → HMS 6 개(hasHydrometeorologicalState), ② HMS 6 개 → EVI:KW:1003110(supportedBy)을 코드가 같은 것끼리 잇는다.
- 결과: 4 개 댐 × 6 개 변수로 hasHydrometeorologicalState 24 개, supportedBy 24 개를 잇는다.

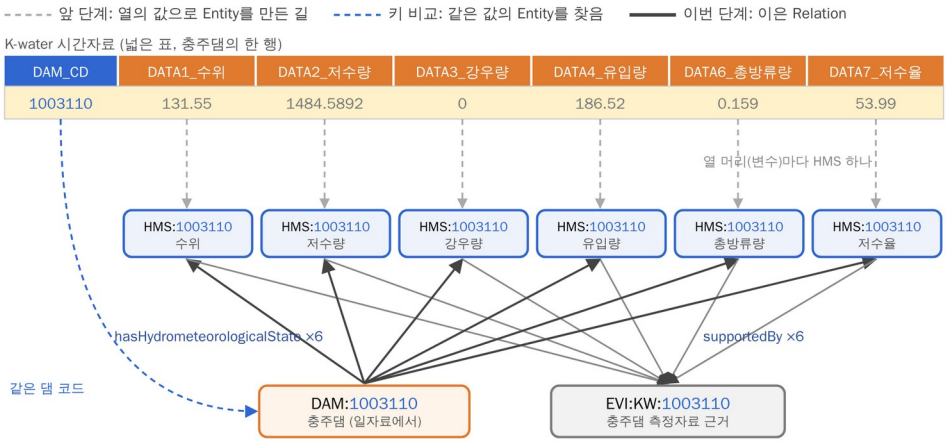



그림 4. 댐 코드로 잇는 Relation — 같은 코드 1003110 을 가진 Dam, HMS, 측정자료 근거를 잇는다

## 2.6 별표 3 댐 이름으로 잇는 Relation

- 그림 5 의 위쪽은 별표 3 에서 추출한 중간 파일 criterion_list.csv 의 한 줄이고, 앞 단계에서 이 줄로 CRI:CR-02 를 만들었다.
- 별표 3 에는 댐 코드가 없어 ① 댐 이름 "충주댐"을 대응 표에서 찾아 1003110 을 얻고, ② CRI:CR-02 → DAM:1003110(appliesToDam)을 쓴다. 이름으로 찾았다는 표시(name-match)를 Relation 에 남긴다.
- ③ 규정이 하나라 근거 Entity 도 EVI:LAW:별표 3 하나이고, 기준 3 개 모두 이것과 잇는다(supportedBy).
- 결과: appliesToDam 3 개(충주, 소양강, 횡성), supportedBy 3 개를 잇는다.

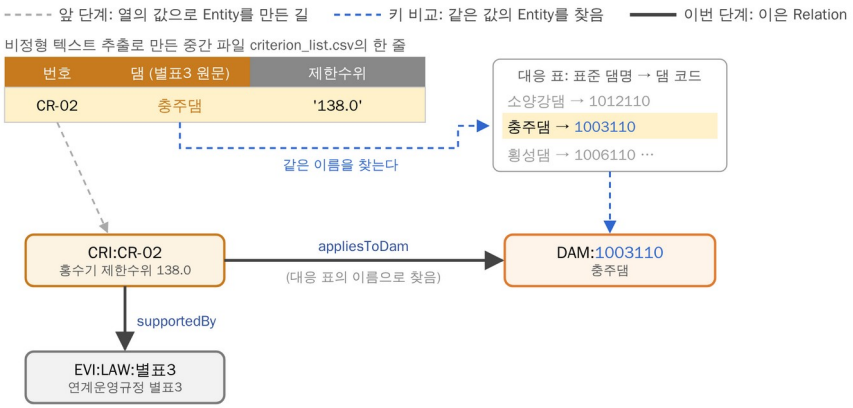



그림 5. 별표 3 댐 이름으로 잇는 Relation — 이름을 대응 표로 댐 코드에 맞춘다

## 2.7 결과: Relation 을 이은 CSV

- 실제 Relation 418 개가 kgv4_relations.csv 에 한 줄씩 쓰였다(그림 6). 한 줄은 시작 Entity(subject), Relation 이름(relation), 끝 Entity(object), 끝 Entity 를 찾은 방법의 이름(rule), 원본의 키로 바로 이었는지 표시(origin), 설명(note)이다.
- 이로써 KG 에 들어갈 내용이 CSV 파일 세 개(kgv4_entities, kgv4_properties, kgv4_relations)에 모두 담겼다.
- 연결 확인: 4 개 댐 전체에서 승인 80 개 모두 댐과 근거에, 운영행위 68 개 모두 댐·승인·근거에, HMS 24 개 모두 댐과 근거에, 기준 3 개 모두 댐과 근거에 이어졌다. 승인 중 운영행위로 이어진 것은 63 개이며, 나머지 17 개는 변경 승인이다.

kgv4_relations.csv — 전체 418줄 중 충주댐의 Entity 몇 개(승인 504, 운영행위 504:1, 수위 HMS, 기준 CR-02, 댐)가 시작인 줄 14개

| subject (시작 Entity) | relation | object (끝 Entity) | rule | origin | note |
|---|---|---|---|---|---|
| DAM:1003110 | hasHydrometeorologicalState | HMS:1003110:수위 | MR-HMS | real | |
| HMS:1003110:수위 | supportedBy | EVI:KW:1003110 | MR-SUP-HMS | real | |
| DAM:1003110 | hasHydrometeorologicalState | HMS:1003110:저수량 | MR-HMS | real | |
| DAM:1003110 | hasHydrometeorologicalState | HMS:1003110:강우량 | MR-HMS | real | |
| DAM:1003110 | hasHydrometeorologicalState | HMS:1003110:유입량 | MR-HMS | real | |
| DAM:1003110 | hasHydrometeorologicalState | HMS:1003110:총방류량 | MR-HMS | real | |
| DAM:1003110 | hasHydrometeorologicalState | HMS:1003110:저수율 | MR-HMS | real | |
| APR:504 | supportedBy | EVI:AP:504 | MR-SUP-APR | real | |
| APR:504 | concernsDam | DAM:1003110 | MR-APR-DAM | real | 승인 레코드의 관측소코드 = 댐 코드 (DAM_CD) |
| OPR:504:1 | performedOnDam | DAM:1003110 | MR-OPR-DAM | derived | 승인 행의 시설 코드 |
| APR:504 | authorizes | OPR:504:1 | MR-AUTH | constructed | 같은 승인 행의 비고에서 분류한 행위 (연구용 구성, 실행 확인 아님) |
| OPR:504:1 | supportedBy | EVI:AP:504 | MR-SUP-OPR | derived | |
| CRI:CR-02 | appliesToDam | DAM:1003110 | MR-CRI-DAM | name-match | 별표3 시설명 일치(코드 없음) |
| CRI:CR-02 | supportedBy | EVI:LAW:별표3 | MR-SUP-CRI | real | |
| … 나머지 404줄 | | | | | |

그림 6. kgv4_relations.csv 의 실제 줄 — 충주댐 Entity 몇 개가 시작인 줄 14 개

# 3. 저장

## 3.1 따른 원리와 진행 방식

- 따른 원리(Huang 외, 2026; 부록 D.2): 모은 자료를 Python 스크립트가 읽어 Cypher 문을 만들고 Neo4j 에 한꺼번에 넣는다. 우리는 JSON 대신 2 장까지 만든 CSV 세 개를 읽어 Cypher 문을 쓰고, 그 문을 모은 파일을 Neo4j 에서 실행했다.
- 따른 원리(Zhou 외, 2023; 부록 D.3): 데이터는 원래 저장소에 두고 키로 KG 와 대응시켜, 질의할 때 그 행을 읽어 온다. 우리는 시간별 측정값과 규정 원문을 KG 밖 CSV 에 두고, KG 에는 그 행을 찾을 키만 두었다. 출처 파일(kgv4_provenance)도 CSV 로 둔다.
- 진행 방식: Neo4j 는 Entity 와 Relation 을 그래프(점과 선)로 저장하는 데이터베이스이고, Cypher 는 Neo4j 에 넣고 찾을 때 쓰는 명령 문장이다. CSV 는 줄의 목록이라 관계를 따라가는 질의를 바로 할 수 없어서, Entity, Relation 순서로 Cypher 문을 써서 Neo4j 에 넣는다.

## 3.2 CSV → Cypher → Neo4j

### 과정 1. Entity 를 Cypher 문으로 쓰고 Neo4j 에 넣기

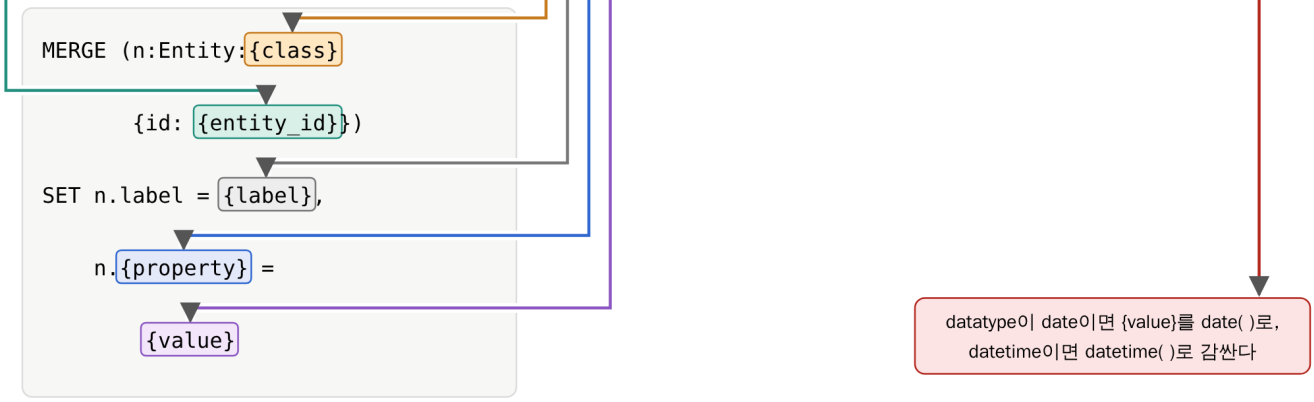
- 원리: Entity 하나가 Cypher 문 하나가 되고, 실행하면 Neo4j 에 Entity 하나가 Class 와 Data Property 값을 가진 채 생긴다.
- 그림 7 의 맨 위는 kgv4_entities.csv 와 kgv4_properties.csv 의 APR:504 줄이며, 화살표를 따라 각 열의 값이 가운데 틀의 { } 자리에 들어간다. datatype 이 date 이면 값을 date( )로 감싸 날짜로 넣어, "2020 년 8 월 안"처럼 기간으로 비교해 찾을 수 있게 한다.
- MERGE 는 Entity 가 없을 때만 만들어서, 같은 문을 다시 실행해도 APR:504 가 둘이 되지 않는다. 264 개 Entity 마다 이런 문이 하나씩 쓰인다.

kgv4_entities.csv (APR:504 줄)

| entity_id | class | label |
|---|---|---|
| APR:504 | Approval | 승인 504 (충주댐) |

kgv4_properties.csv (APR:504 줄 2개)

| entity_id | property | value | datatype |
|---|---|---|---|
| APR:504 | approvalTime | 2011-06-27 | date |
| APR:504 | approvalContent | 방류시작 2011-06-27 18:00;<br>접수방류량 2000㎥/s;<br>비고: (실제방류는 20시부터) | string |




{ } 자리에 CSV 열의 값이 들어간다. 맨 아래 두 줄(n.{property} = {value})은 properties의 줄마다 생긴다.

↓ APR:504의 줄 3개를 넣으면 (실제 파일의 한 줄)

```
MERGE (n:Entity:Approval {id: 'APR:504'})
SET n.label = '승인 504 (충주댐)',
    n.approvalTime = date('2011-06-27'),
    n.approvalContent = '방류시작 2011-06-27 18:00; …';
```

MERGE — 이 Entity가 없으면 만들고 있으면 그대로 쓴다. ( ) 하나가 Entity 하나이고 n은 이 문 안에서 부르는 이름이다.
:Entity — 모든 Entity에 붙는 공통 표시로, Class를 몰라도 ID 하나로 찾게 한다. SET — Data Property 값을 넣는다.

↓ Neo4j에서 실행하면

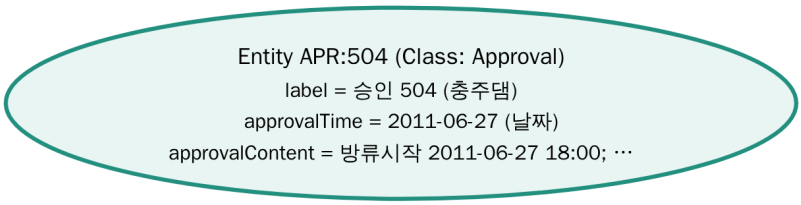



그림 7. 과정 1 — CSV 열의 값이 Cypher 문의 자리로 들어가고, 실행하면 Neo4j 에 Entity 가 생긴다 (APR:504)

## 과정 2. Relation 을 Cypher 문으로 쓰고 Neo4j 에 넣기

- 원리: Relation 하나가 Cypher 문 하나가 된다. 문은 Neo4j 에 이미 있는 두 Entity 를 찾고 그 사이에 Relation 을 만든다.
- 그림 8 의 맨 위는 kgv4_relations.csv 의 한 줄이다. Relation 은 Neo4j 안의 두 Entity 사이에 긋는 것이라, 과정 1 의 Entity 문이 먼저 실행되어 있어야 한다. 418 개 Relation 마다 이런 문이 하나씩 쓰인다.

kgv4_relations.csv (한 줄 = Relation 하나)

| subject | relation | object | rule | origin | note |
|---|---|---|---|---|---|
| APR:504 | concernsDam | DAM:1003110 | MR-APR-DAM | real | 승인 레코드의 관측소코드 = 댐 코드(DAM_CD) |

```
MATCH (a:Entity {id: {subject}}),
      (b:Entity {id: {object}})
MERGE (a)-[r:{relation}]->(b)
SET r.rule = {rule},
    r.origin = {origin},
    r.note = {note}
```

CSV에서 오는 것은 찾을 ID 값뿐이다

subject, object 열의 값(APR:504, DAM:1003110)이 {subject}, {object} 자리에 들어가고, MATCH가 Neo4j에 이미 들어 있는 Entity 가운데 그 ID를 가진 것을 찾는다.

↓ 이 줄을 넣으면 (실제 파일의 한 줄)

```
MATCH (a:Entity {id: 'APR:504'}), (b:Entity {id: 'DAM:1003110'})
MERGE (a)-[r:concernsDam]->(b)
SET r.rule = 'MR-APR-DAM', r.origin = 'real',
    r.note = '승인 레코드의 관측소코드 = 댐 코드(DAM_CD)';
```

MATCH — Neo4j에 이미 들어 있는 Entity 가운데 ID가 APR:504, DAM:1003110인 것을 찾아 a, b로 부른다.
MERGE (a)-[r:concernsDam]->(b) — a에서 b로 가는 Relation r이 없을 때만 만든다.
SET — Relation r에 rule, origin, note 열의 값을 넣는다.

↓ Neo4j에서 실행하면

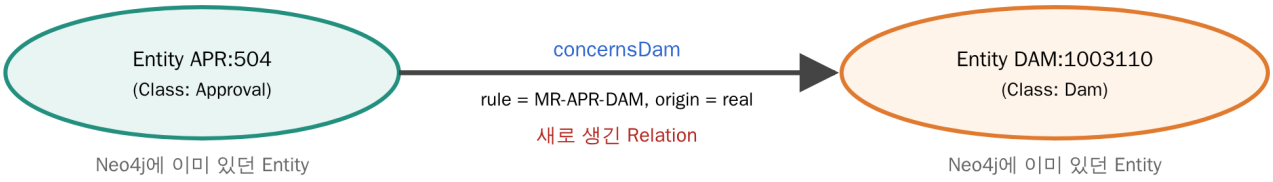



그림 8. 과정 2 — 관계 CSV 한 줄이 Cypher 문이 되고, 실행하면 두 Entity 사이에 Relation 이 생긴다

## 과정 3. Cypher 문을 모아 Neo4j 에서 실행하기

- 원리: 과정 1·2 의 문을 파일 하나(kgv4_load.cypher, 684 줄)에 모아 Neo4j 가 위에서부터 한 번에 실행한다. Relation 문은 Entity 를 찾아야 하므로 Entity 문 뒤에 둔다(그림 9 왼쪽).
- 실행할 때는 Neo4j 를 비운 뒤 파일 전체를 다시 넣는다(오른쪽). MERGE 는 없는 것을 더할 뿐 CSV 에서 지운 줄을 Neo4j 에서 지우지 않으므로, 비우지 않으면 CSV 에서 지운 Relation 이 Neo4j 에 남는다. 마지막에 센 수(264, 418)가 CSV 의 줄 수와 같으면 Neo4j 의 KG 가 CSV 와 같다.

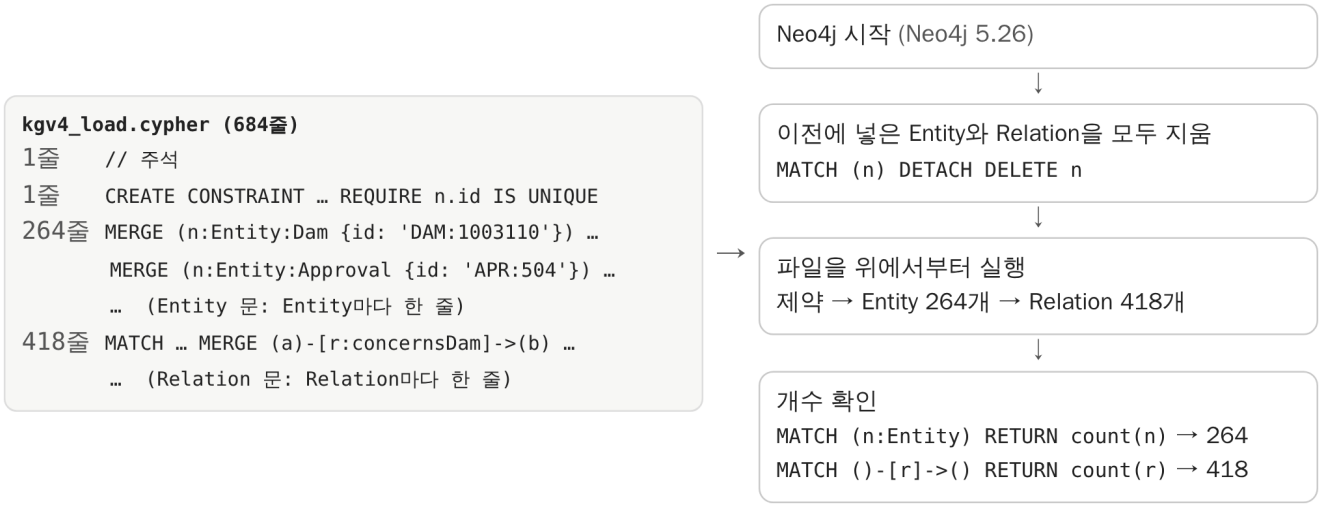



그림 9. 과정 3 — Cypher 파일의 구성과 실행 순서

## 3.3 KG 밖에 두는 것: 측정 저장소와 문서 저장소

- 시간별 측정값(508,246 행)과 규정 원문은 CSV 로 두고, KG 에는 그 행을 찾을 키만 Data Property 로 둔다. 조회할 때 코드가 이 키로 CSV 의 행을 읽는다.
- 그림 10 의 위는 측정 저장소이다. HMS:1003110:수위의 stationCode 와 찾을 시각으로 DAM_CD 와 SDATE_raw 가 같은 행을 찾으면, DATA1_수위 열이 그 시각의 수위(139.55 EL.m)이다. 아래는 문서 저장소이며, EVI:LAW: 별표 3 의 sourceLocator 로 규정 원문의 행을 찾는다.

측정 저장소 — KG의 Entity가 가진 값(stationCode, 시각)으로 CSV의 행을 찾는다

Entity HMS:1003110:수위
stationCode = 1003110
variableType = 수위

→ DAM_CD = stationCode, SDATE_raw = 찾는 시각(2020-08-02 18:00 → 2020080218), 열 = variableType이 든 열(DATA1_수위)

kwater_mywater_hydr_H_aligned_wide.csv.gz (2010-08~2020-12) · kwater_mywater_hydr_H_wide.csv.gz (2021-01~2026-09)도 같은 열 · 4개 댐 합계 508,246행

| source_file | retrieval_time_utc | param1_resolution | DAM_CD | SDATE_raw | DATA1_수위(EL.m) | DATA2_저수량(MCM) | DATA3_강우량(mm) | DATA4_유입량(CMS) | DATA5_source_label_unknown | DATA6_총방류량(CMS) | DATA7_저수율(%) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json | 2026-10-09T01:36:21Z | H | 1003110 | 2020080217 | 139.37 | 2111.215 | 0.711 | 4693.654 | 4693.654 | 707.265 | 76.8 |
| 01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json | 2026-10-09T01:36:21Z | H | 1003110 | 2020080218 | 139.55 | 2126.472 | 1.901 | 4945.201 | 4945.201 | 707.145 | 77.3 |
| 01_raw/KWater/mywater_aligned/hydr/H/1003110/2020-07-09_2020-08-07.json | 2026-10-09T01:36:21Z | H | 1003110 | 2020080219 | 139.69 | 2138.381 | 0.273 | 4015.509 | 4015.509 | 707.453 | 77.8 |

문서 저장소 — KG의 Entity가 가진 값(sourceLocator)으로 CSV의 행을 찾는다

Entity EVI:LAW:별표3
sourceLocator = 별표 3

→ sourceLocator 열이 "별표 3"인 행의 text가 규정 원문이다

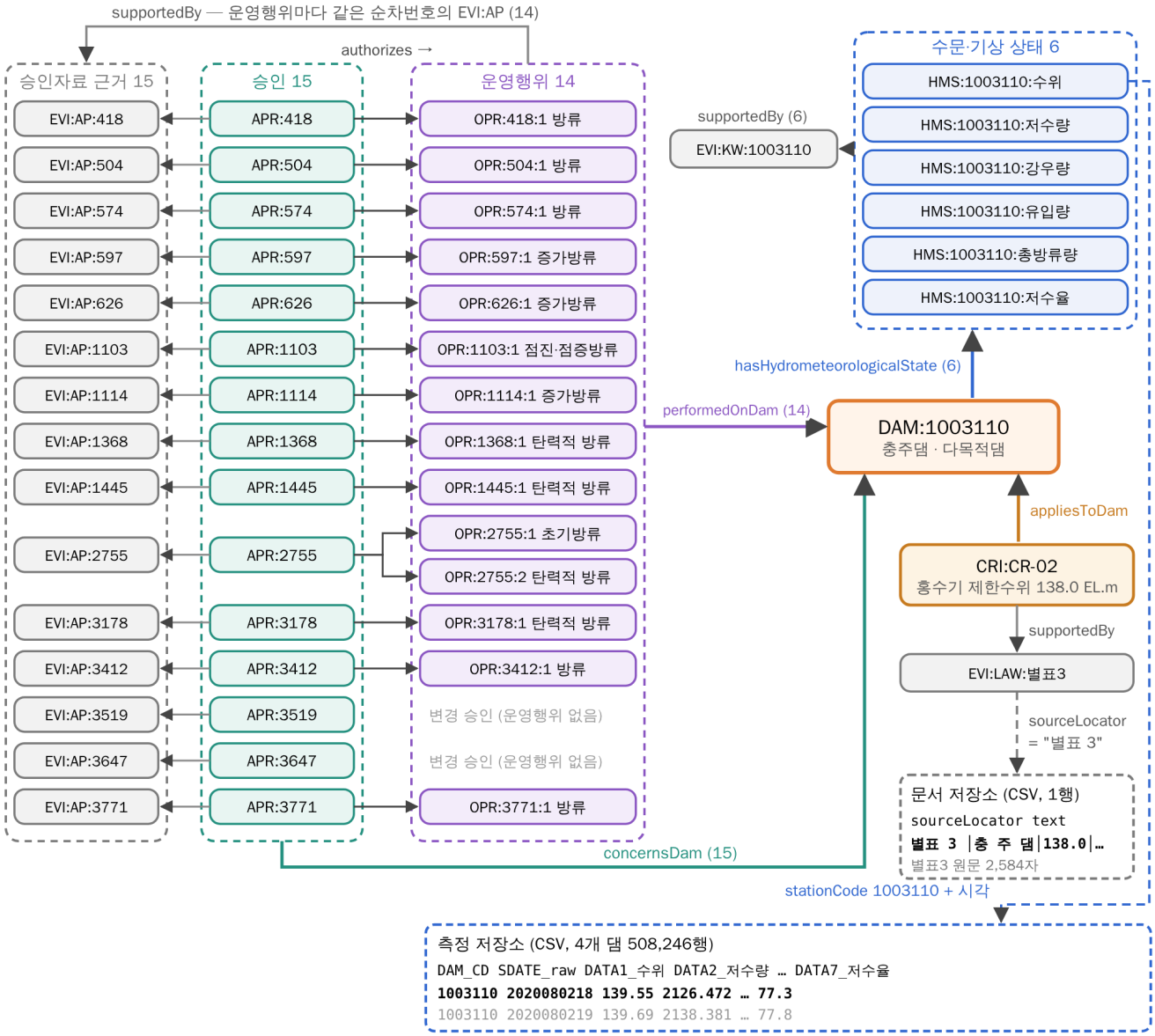
kgv4_documents.csv (1행)

| sourceLocator | doc_title | text (2,584자 중 앞부분) |
|---|---|---|
| 별표 3 | 댐과 보 등의 연계운영규정 (기후에너지환경부 훈령 제42호, 시행 2026-07-08) | 댐의 홍수기 제한수위(제6조)<br>[별표 3] 댐의 홍수기 제한수위(제6조)<br>┌──┬──┬──┬──┐<br>\|구 분 \|댐 종류 \|댐 명 \|홍수기 제한수위 (EL.m) \|시설별 최저 운영수위 (EL.m) \|<br>├──┼──┼──┼──┤<br>\|한 강 수 계 \|다목적댐 \|소양강댐 \|190.3 \|- \|<br>\| \| \|충 주 댐 \|138.0 \|- \|<br>\| \| \|횡 성 댐 \|178.2 \|- \|<br>\| \|수력발전댐\|화 천 댐 \|175.0 \|- \|<br>\| \| \|춘 천 댐 \|102.0 \|98.0 \|<br>\| \| \|의 암 댐 \| 70.5 \|67.0 \|<br>… |

그림 10. 외부 저장소 CSV 의 실제 모습과, KG 의 값으로 행을 찾는 방법

## 3.4 결과: Neo4j 에 저장된 KG

- Neo4j 에 Entity 264 개와 Relation 418 개가 저장되었다. 그림 11 은 그중 충주댐의 Entity 53 개와 Relation 이다.
- 왼쪽 세 점선 상자는 승인자료 근거 15, 승인 15, 운영행위 14 개이며 한 줄이 승인 한 건이다. 3519 와 3647 은 변경승인이라 운영행위가 없다.




실선은 Neo4j에 저장된 Relation이고, 점선 상자(승인, 운영행위, 근거, HMS) 안의 Entity는 각각 그 Relation을 가진다(괄호 안은 개수). 맨 아래의 점선 화살표는 Relation이 아니라, KG의 값으로 외부 저장소의 행을 찾아오는 길이다.

그림 11. Neo4j 에 저장된 KG(충주댐 Entity 53 개)와 외부 저장소로 가는 길

# 4. 검색·답변

## 4.1 따른 원리와 진행 방식

- 따른 원리(Purohit 외, 2024; 부록 D.1): KG 를 조회한 결과를 LLM 에 근거로 주어 답변을 만들어, LLM 이 자료에 없는 사실을 지어내는 것(환각)을 줄인다. 우리는 이번 시험에서 조회 템플릿을 CQ 마다 미리 정해 두고, LLM 은 질문에서 의도(CQ1~CQ6)와 값(댐, 기간)만 뽑게 했다.
- 진행 방식: 의도·값 추출(LLM) → 입력 확인(코드) → KG 검색과 외부 저장소 조회(코드) → 근거 묶음(코드) → 답변(LLM)의 다섯 과정이다. LLM 은 처음과 마지막에만 쓴다(같은 모델 Qwen3-32B-AWQ, 지시문만 다름).
- 아래에서는 서버에서 실행한 질문 하나(2026-10-16)를 따라간다.

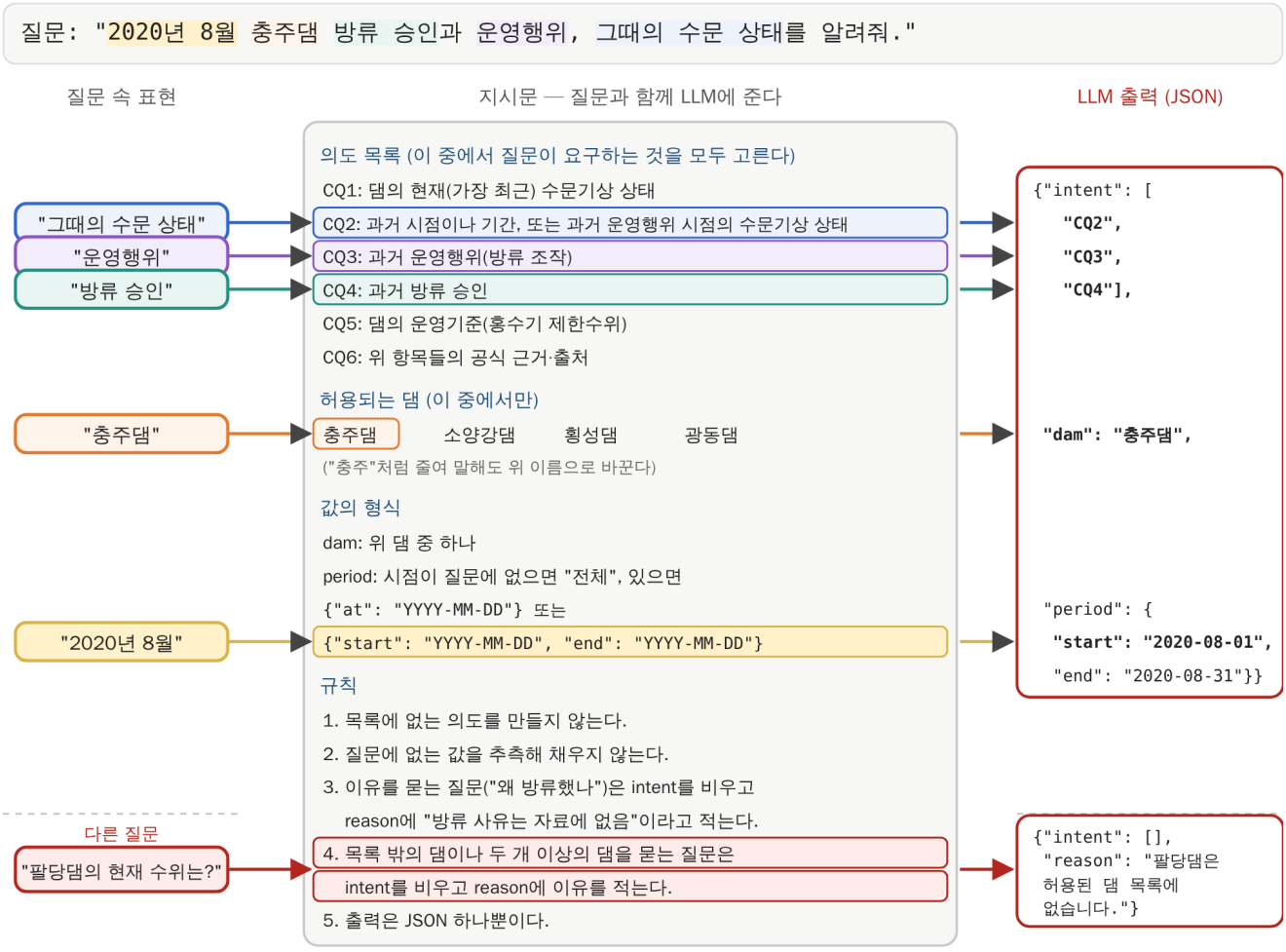
## 4.2 질문에서 근거 묶음까지

### 입력

- 질문: "2020 년 8 월 충주댐 방류 승인과 운영행위, 그때의 수문 상태를 알려줘."

### 과정 1. 의도·값 추출 (LLM)

- 원리: LLM 은 지시문(그림 12 가운데)에 정해 둔 의도 목록과 허용되는 댐 안에서만 고르고, 값은 Cypher 템플릿에 그대로 넣을 수 있는 형식으로 적는다.
- 화살표처럼 "그때의 수문 상태"는 CQ2, "운영행위"는 CQ3, "방류 승인"은 CQ4 가 되고, "2020 년 8 월"은 2020-08-01 ~ 2020-08-31 이 된다. 아래의 "팔당댐의 현재 수위는?"는 팔당댐이 허용되는 댐에 없어 규칙 4 에 따라 intent 가 비고, KG 를 조회하지 않는다.




LLM 출력 원문: `{"intent": ["CQ2", "CQ3", "CQ4"], "dam": "충주댐", "period": {"start": "2020-08-01", "end": "2020-08-31"}}`

그림 12. 의도·값 추출 — 질문 속 표현을 지시문의 의도 목록, 허용되는 댐, 값의 형식에 맞춰 JSON 으로 적는다

### 과정 2. 입력 확인 (코드)

- 의도가 CQ1~CQ6 안인지, 댐이 4 개 댐 안인지 확인하고 이름을 표준 댐명으로 바꾼다("충주" → 충주댐).
- 기간의 끝 날짜는 그 날 전체로 읽어 [2020-08-01 00:00, 2020-09-01 00:00)으로 바꾼다. 확인을 통과하지 못하면 조회하지 않고 "질의 불가: 이유"를 답변 과정으로 넘긴다.

### 과정 3. KG 검색과 외부 저장소 조회 (코드)

- 원리: 고른 의도마다 미리 정한 Cypher 템플릿에 댐 이름과 기간만 넣어 실행한다. 템플릿은 KG 의 Relation 을 따라가고, 측정값과 원문은 KG 의 키로 외부 저장소에서 가져온다.

- 그림 13 에서 댐 코드 템플릿은 hasHydrometeorologicalState 를 따라 stationCode 1003110 을 얻는다. CQ4 템플릿은 concernsDam 으로 이어진 승인 15 개 가운데 기간 안의 APR:3412, APR:3519, APR:3647 을, CQ3 템플릿은 performedOnDam 으로 이어진 운영행위 가운데 OPR:3412:1 과 그것을 허가한 APR:3412(authorizes) 를 찾는다.
- CQ2 는 운영행위의 시각마다 측정 저장소에서 stationCode 와 operationTime(2020080218)이 같은 행을 읽는다. 정각이 아닌 시각은 앞뒤 정시 두 행을 읽는다.
- 다른 의도가 있는 질문은 표 2 의 경로를 따른다.

입력 확인을 거친 의도 CQ2 · CQ3 · CQ4 댐 = 충주댐 기간 = 2020-08-01 00:00부터 2020-09-01 00:00 전까지

댐 코드 템플릿 (모든 의도)

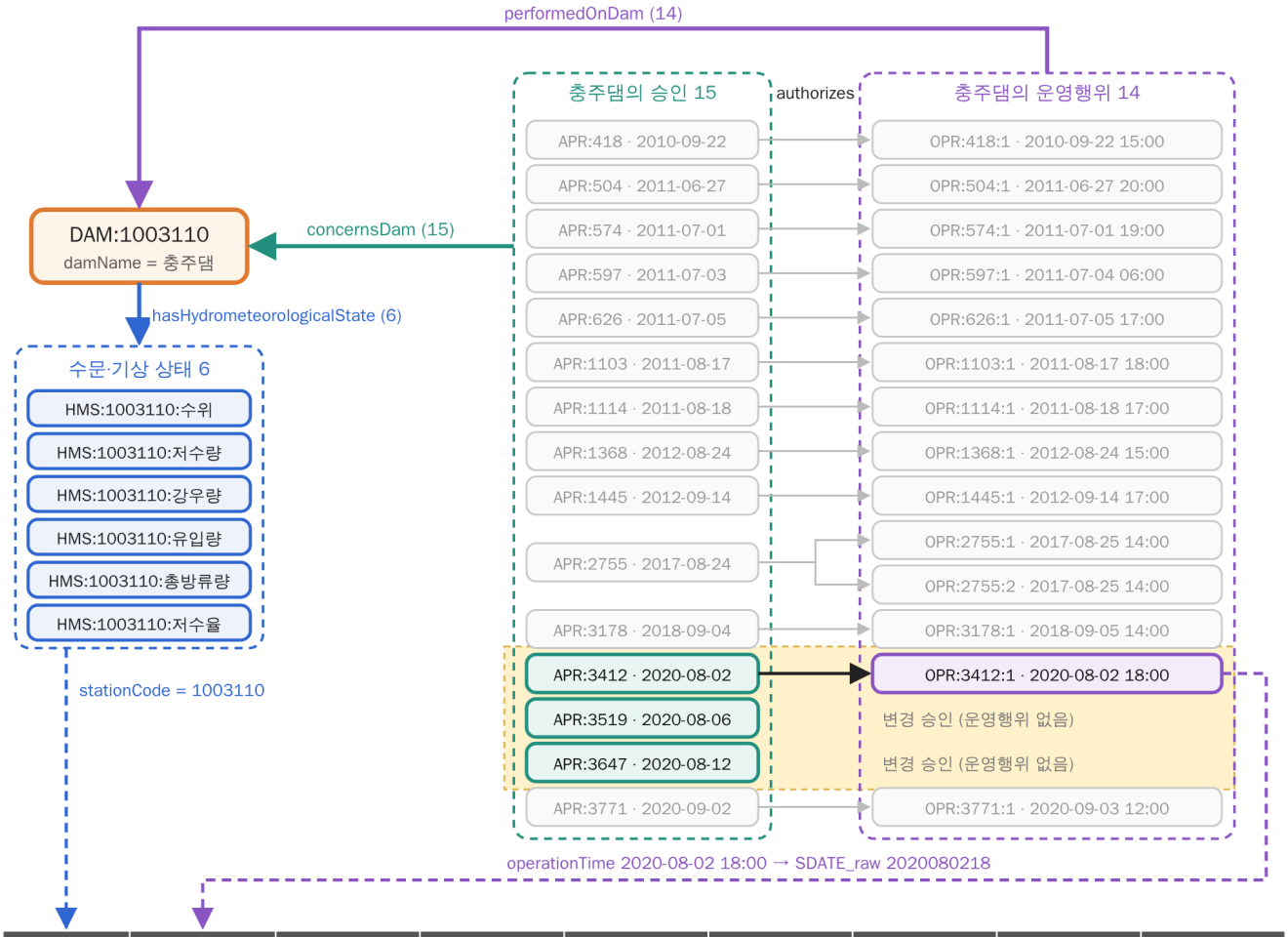
```
MATCH (d:Dam {damName:'충주댐'})
  -[:hasHydrometeorologicalState]->(h)
RETURN h.id, h.variableType,
       h.stationCode
```

CQ4 승인 템플릿

```
MATCH (a:Approval)-[:concernsDam]->
      (d:Dam {damName:'충주댐'})
WHERE a.approvalTime >= date('2020-08-01')
  AND a.approvalTime <  date('2020-09-01')
RETURN a.id, a.approvalTime,
       a.approvalContent
```

CQ3 운영행위 템플릿 (CQ2도 씀)

```
MATCH (o:Operation)-[:performedOnDam]->
      (d:Dam {damName:'충주댐'})
WHERE o.operationTime >=
        datetime('2020-08-01T00:00:00')
  AND o.operationTime <
        datetime('2020-09-01T00:00:00')
OPTIONAL MATCH
  (a:Approval)-[:authorizes]->(o)
RETURN o.id, o.operationType,
       o.operationTime, a.id
```




| DAM_CD | SDATE_raw | DATA1_수위 (EL.m) | DATA2_저수량 (MCM) | DATA3_강우량 (mm) | DATA4_유입량 (CMS) | DATA5_source_ label_unknown | DATA6_총방류량 (CMS) | DATA7_저수율 (%) |
|---|---|---|---|---|---|---|---|---|
| 1003110 | 2020080217 | 139.37 | 2111.215 | 0.711 | 4693.654 | 4693.654 | 707.265 | 76.8 |
| **1003110** | **2020080218** | **139.55** | **2126.472** | **1.901** | **4945.201** | **4945.201** | **707.145** | **77.3** |
| 1003110 | 2020080219 | 139.69 | 2138.381 | 0.273 | 4015.509 | 4015.509 | 707.453 | 77.8 |

측정 저장소 kwater_mywater_hydr_H_aligned_wide.csv.gz의 행 (앞의 source_file 등 3열은 생략). 고른 행의 값이 [MEA:1003110:2020080218]이 된다.

실선은 Neo4j에 저장된 Relation, 템플릿의 주황·노랑 글자는 입력 확인을 거친 댐 이름과 기간이 들어간 자리이다. 노란 띠는 기간 안의 Entity이다. 진하게 칠한 Entity가 이 질문에서 찾은 것이고 흐린 것은 기간 밖이라 걸러진 것이다. 점선은 KG의 값으로 측정 저장소의 행을 찾아가는 길이다.

그림 13. KG 검색과 외부 저장소 조회 — 템플릿이 따라가는 Relation 과 기간 조건, KG 의 값으로 찾는 측정 저장소의 행

### 표 2. 의도별 Cypher 템플릿이 따라가는 Relation 과 외부 저장소

| 의도 | 따라가는 Relation (Cypher 템플릿) | 외부 저장소 |
|---|---|---|
| CQ1 현재 상태 | Dam → hasHydrometeorologicalState → HMS (stationCode) | 측정: 저장된 가장 최근 시각의 행 |
| CQ2 과거 상태 | (CQ3 과 함께면) 운영행위의 operationTime | 측정: 그 시각의 행, CQ3 이 없으면 기간 안의 행 |
| CQ3 운영행위 | Operation → performedOnDam → Dam, Approval → authorizes → Operation | — |
| CQ4 승인 | Approval → concernsDam → Dam | — |
| CQ5 운영기준 | Criterion → appliesToDam → Dam, Criterion → supportedBy → EvidenceSource | 문서: sourceLocator 의 원문 |
| CQ6 근거 | 조회한 Entity → supportedBy → EvidenceSource | — |

## 과정 4. 근거 묶음 (코드)

- 조회 결과를 의도별 구역으로 나누고, 항목마다 출처 번호를 붙인다. 출처 번호는 KG 의 Entity ID 이고, 측정값은 [MEA:댐 코드:시각]이다.
- 값은 저장소의 원값 그대로 쓰고 계산한 값은 넣지 않는다. 끝에 근거 사용 원칙 3 문장을 붙인다.

```
[질문] 2020년 8월 충주댐 방류 승인과 운영행위, 그때의 수문 상태를 알려줘.
[대상] 충주댐, 기간 2020-08-01 ~ 2020-08-31

[근거 1. 운영행위와 허가한 승인]  (CQ3) 운영행위의 시각은 비고에 적힌 시각, 없으면 승인된 방류 시작 시각이다.
[OPR:3412:1] 방류, 시각 2020-08-02 18:00 ← 승인 [APR:3412]

[근거 2. 운영 시각의 상태]  (CQ2)
[MEA:1003110:2020080218] 2020-08-02 18:00  수위 139.55 EL.m, 저수량 2126.472 MCM, 강우량 1.901 mm,
   유입량 4945.201 CMS, 총방류량 707.145 CMS, 저수율 77.3 %

[근거 3. 방류 승인]  (CQ4)
[APR:3412] 승인일 2020-08-02: 방류시작 2020-08-02 18:00; 접수방류량 3000㎥/s
[APR:3519] 승인일 2020-08-06: 방류시작 2020-08-02 18:00; 접수방류량 7000㎥/s; 비고: 방류기간 (당초) … (변경)
…
[APR:3647] 승인일 2020-08-12: 방류시작 2020-08-06 06:00; 접수방류량 7000㎥/s; 비고: 수문방류 변경승인(방류기
간연장) …

[근거 사용 원칙] 근거에 있는 내용만 사실로 쓰고 출처 번호를 붙인다 / 비교·해석은 판단임을 밝힌다 / 없으면 "자료에 없
음"
```

의도별 구역 출처 번호 = KG의 Entity ID, 측정값은 MEA 원칙 3문장

그림 14. 근거 묶음 — LLM 에 주는 근거 전체

## 과정 5. 답변 (LLM)

- LLM 에 답변 지시문과 근거 묶음을 준다. 지시문: 근거 묶음에 있는 내용만 사실로 쓰고 출처 번호를 붙인다, 비교·해석은 "판단"으로 나누고 쓴 근거를 적는다, 없으면 "자료에 없음".
- 주의 사항도 함께 준다: 운영행위 시각은 비고의 시각이거나 승인된 방류 시작 시각이다, "현재"는 저장된 가장 최근 시각이며 실시간이 아니다, 방류 사유는 자료에 없으므로 단정하지 않는다.

## 4.3 결과: 답변

- 그림 15 가 이 질문의 답변이다. "근거"에는 승인 세 건의 방류량과 기간이 [APR:3412], [APR:3519], [APR:3647] 과 함께, 방류 시작 시각의 수문 상태가 [MEA:1003110:2020080218]과 함께 쓰여 있다.
- "판단"에는 방류량이 3000㎥/s 에서 7000㎥/s 로 늘고 기간이 연장되었다는 비교가 쓴 근거와 함께 있다. 빨간 글씨 문장은 자료에 없는 사유를 추정한 것으로, 지시문의 "사유를 단정하지 않는다"를 어겼다.
- 근거 묶음의 운영행위 [OPR:3412:1]은 답변에 출처 번호로 쓰이지 않았다.

```
근거
- 2020년 8월 2일 18:00에 방류가 승인되었으며, 방류량은 3000㎥/s로 접수되었습니다. [APR:3412]
- 이후 8월 6일에 방류 승인이 변경되어 방류량이 7000㎥/s로 증가하고, 방류 기간이 8월 2일 18:00부터 8월 16일 24:00까지로
  확정되었습니다. [APR:3519]
- 8월 12일에는 방류 기간이 다시 연장되어 8월 6일 06:00부터 8월 22일 00:00까지로 확정되었습니다. [APR:3647]
- 방류가 시작된 2020년 8월 2일 18:00 기준, 수위는 139.55 EL.m, … 저수율은 77.3%입니다.
[MEA:1003110:2020080218]

판단
- 방류량은 처음 3000㎥/s에서 이후 7000㎥/s로 증가했습니다. 방류 기간은 여러 차례 변경되며 최종적으로 8월 22일 00:00까지
  연장되었습니다. (근거: [APR:3412], [APR:3519], [APR:3647])
- … 이는 댐의 수문 상황과 유입량 변화에 따라 조정된 것으로 판단됩니다. (근거: [APR:3412], …)
```

사실마다 출처 번호 판단마다 쓴 근거 빨간 글씨: 자료에 없는 사유를 추정한 문장 (지시문의 "사유를 단정하지 않는다"를 어김)

그림 15. 답변 — 근거(출처 번호)와 판단(쓴 근거)으로 나뉜다

# 5. 검토할 점과 다음 계획

## 1. 답할 수 있는 질문의 범위 (온톨로지, 검색·답변)

- 온톨로지는 CQ1~CQ6 에 답할 수 있는 구조로 만들었고, 검색은 CQ 마다 정한 Cypher 템플릿으로 한다(4 장). 그래서 지금 구조는 질문의 의도가 CQ1~CQ6 에 해당할 때만 답할 수 있다.
- 의도 목록에 없는 질문은 과정 1 에서 intent 가 비어 조회하지 않는다. 예: 방류 사유를 묻는 질문, 4 개 댐 밖의 댐이나 두 댐을 함께 묻는 질문.

## 2. 과제 발표와 시연

- 11/20 과제 발표와 11/26 시연은 지금 구조(4 장)로 한다.

## 3. 연구

- 온톨로지 생성 방식을 다시 검토한다.
- 검색·답변과 검증에 대한 선행 논문을 더 찾아 읽고 지금 방법과 비교한 뒤, 더 좋은 결과를 내는 방법을 적용한다. 검증은 11/02 회의에 따라 다른 연구들이 KG·온톨로지를 어떻게 검증했는지부터 조사한다.

# 부록 A. 용어 정리

## 표 3. 용어 정의

| 용어 | 본 연구에서의 정의 |
|---|---|
| **Relation 잇기** | 온톨로지의 Relation 정의에 맞는 Class 사이의 조합 가운데, 키로 실제로 이어질 Entity 쌍(시작 Entity → 끝 Entity)을 찾아 잇는 일. |
| **키** | 두 자료나 두 Entity 가 같은 대상에 관한 것임을 알려 주는 값. 예: 승인 순차번호 504, 댐 코드 1003110. Relation 을 이을 때와 외부 저장소에서 행을 찾을 때 같은 뜻으로 쓴다. |
| **시작·끝 Entity** | Relation 한 줄의 출발점과 도착점. "APR:504 —concernsDam→ DAM:1003110"에서 APR:504 가 시작 Entity, DAM:1003110 이 끝 Entity. |
| **개체 해소** | 자료마다 다르게 적힌 같은 대상(충주, 충 주 댐, 1003110)을 하나의 기준 식별자에 대응시키는 일. 문헌의 entity disambiguation. |
| **기준 식별자** | 같은 대상을 가리키는 표기들을 모으는 기준 ID. 본 연구는 K-water 댐 코드, GraphAide 는 Wikidata 의 QID. |
| **Neo4j** | Entity 와 Relation 을 그래프(점과 선) 모양으로 저장하는 데이터베이스. |
| **Cypher** | Neo4j 에 Entity 와 Relation 을 넣고 찾을 때 쓰는 명령 문장. MERGE(없으면 만듦), MATCH(찾음), SET(값을 넣음) 등. |
| **지시문** | LLM 에 질문과 함께 주는 지침 글(프롬프트). 의도·값 추출용과 답변용이 따로 있다. |
| **조회 템플릿** | CQ 마다 미리 정해 둔 Cypher 문. 댐 이름과 기간만 바꿔 넣어 실행한다. |

| 용어 | 본 연구에서의 정의 |
|---|---|
| **의도** | 질문이 요구하는 CQ(CQ1~CQ6). 한 질문에 여러 개가 있을 수 있다. |
| **근거 묶음** | KG 와 저장소에서 찾은 근거를 의도별 구역과 출처 번호를 붙여 LLM 에 주는 글. |
| **출처 번호** | 근거 항목마다 붙이는 표시. KG 의 Entity ID 이거나 측정값의 [MEA:댐 코드:시각]. |
| **환각** | LLM 이 자료에 없는 사실을 지어내는 것. |

# 부록 B. Ontology 가 답해야 할 질문과 관계

## 표 4. 세부 질문 (CQ1~CQ6)

| 번호 | 질문 대상 | 세부 질문 |
|---|---|---|
| **CQ1** | 현재 수문·기상 상태 | 대상 댐의 현재 또는 기준시점의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 무엇인가? |
| **CQ2** | 과거 수문·기상 상태 | 선택된 과거 시점 또는 사례에서 대상 댐의 강우, 수위, 유입량, 방류량 등 수문·기상 상태는 어떠했는가? |
| **CQ3** | 과거 운영행위 | 해당 과거 시점 또는 사례에서 실제로 수행된 댐 운영행위는 무엇이었는가? |
| **CQ4** | 과거 방류 승인 | 해당 과거 시점 또는 사례에서 어떤 방류 승인이 있었으며, 그 승인내용은 무엇인가? |
| **CQ5** | 현재 운영기준 | 현재 대상 댐에 적용되는 운영기준은 무엇이며, 현재 운영 판단과 관련된 기준값은 무엇인가? |
| **CQ6** | 공식 근거 | 각 수문·기상 상태, 운영행위, 승인내용 및 운영기준을 확인할 수 있는 공식 자료 또는 근거문서는 무엇인가? |

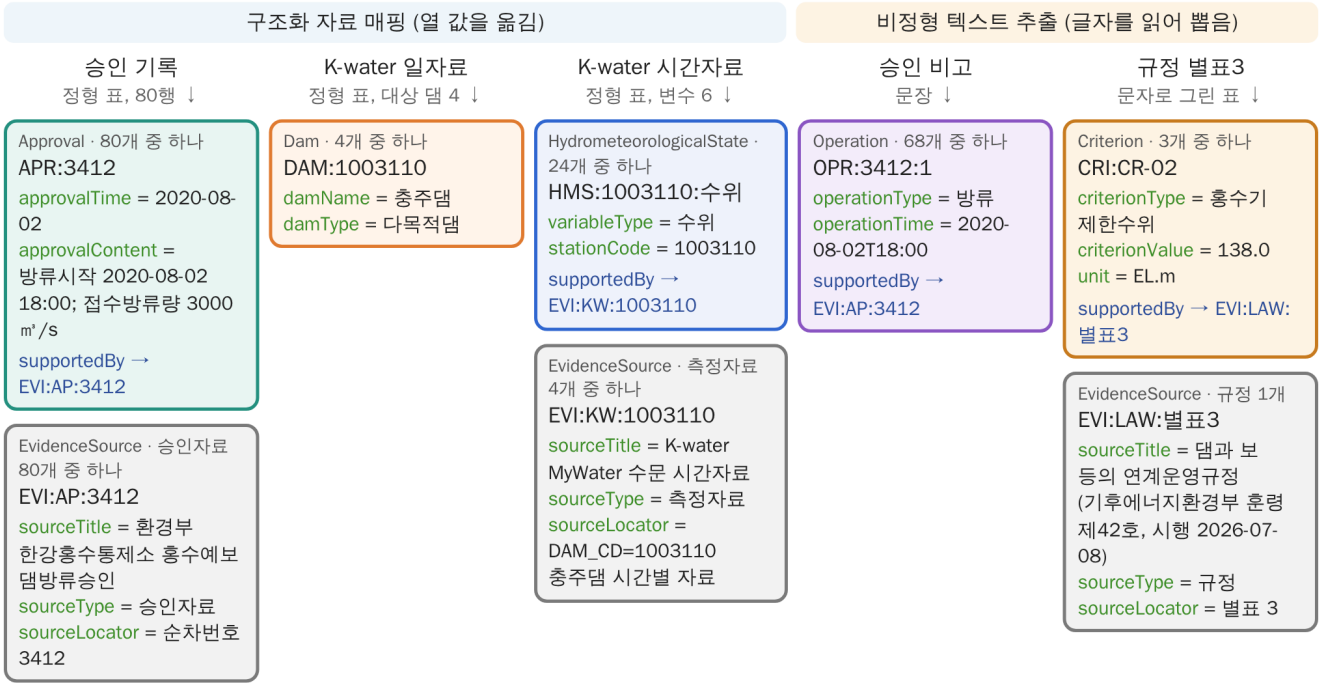
## 표 5. Relation (9 개)

| Subject | Relation | Object | 의미 |
|---|---|---|---|
| **Dam** | hasHydrometeorologicalState | HydrometeorologicalState | 댐이 해당 수문·기상 변수 정보와 연결됨 |
| **Operation** | performedOnDam | Dam | 운영행위가 특정 Dam 에서 수행됨 |
| **Approval** | authorizes | Operation | Approval 이 특정 Operation 을 승인함 |
| **Approval** | concernsDam | Dam | Approval 이 특정 Dam 에 관한 것임 |
| **Criterion** | appliesToDam | Dam | 운영기준이 특정 Dam 에 적용됨 |
| **HydrometeorologicalState** | supportedBy | EvidenceSource | 상태를 확인할 수 있는 근거자료 |
| **Operation** | supportedBy | EvidenceSource | 운영행위를 확인할 수 있는 근거자료 |
| **Approval** | supportedBy | EvidenceSource | 승인을 확인할 수 있는 근거자료 |
| **Criterion** | supportedBy | EvidenceSource | 운영기준을 확인할 수 있는 근거자료 |

# 부록 C. 10/30 자료까지의 결과: 자료 준비, Class·Data Property 채우기

- 자료 준비: 승인 기록 3,929 행에서 중복 7 행을 지우고 대상 4 개 댐의 80 행을 남겼다. K-water 시간자료 두 파일을 이어 붙여 4 개 댐의 508,246 행을 측정 저장소에, 규정 XML 에서 꺼낸 별표 3 원문을 문서 저장소에 두었다.
- Class·Data Property 채우기: 정형 표는 구조화 자료 매핑으로, 승인 비고와 별표 3 은 비정형 텍스트 추출로 Entity 와 속성값을 만들었다. 그림 16 은 충주댐에 관한 실제 Entity 를 자료마다 하나씩 보인 것이다. 결과는 kgv4_entities.csv(Entity 264 개)와 kgv4_properties.csv(속성값 616 개)이다.

- 이 단계에서 이어진 Relation 은 같은 행·같은 자료에서 함께 만든 근거(supportedBy) 175 개뿐이었다. 나머지 Relation 243 개를 2 장에서 이었다.




충주댐에 관한 실제 Entity를 자료마다 하나씩 보인 것이다. 초록 글자가 Data Property, 굵은 글자가 그 값이다. 같은 Class의 다른 Entity도 같은 Data Property를 가진다. 이 단계에서 이어진 Relation은 같은 행·같은 자료에서 함께 만든 근거로 가는 supportedBy뿐이다.

그림 16. 10/30 자료까지 만든 Entity — 자료마다 충주댐의 실제 Entity 하나와 그 Data Property 값

# 부록 D. 근거 문헌의 원리

## D.1 Purohit 외(2024) — Relation 잇기(개체 해소), 검색·답변

- 무엇인가: 여러 출처의 정형·비정형 자료로 KG 를 만들고, 그 KG 를 조회·추론하는 질의 시스템 GraphAide 를 제안한 학회 논문이다(IEEE BigData 2024). OpenAI LLM, Chroma 벡터 DB, Neo4j, LangChain 으로 구현했다.
- KG 생성 단계의 개체 해소: 기준 지식베이스로 DARPA Wikidata(DWD)를 쓰고, QID 마다 설명 문서를 만들어 벡터 DB 에 넣었다. 온톨로지와 DWD 에 기반한 에이전트가 글에서 대상의 표기를 뽑고, RAG 기반 후보 재순위로 가장 가까운 QID 를 고른다. → 기준 식별자에 대응시키는 원리를 적용했다. 대상이 댐 4 개라 이번에는 코드 일치와 대응 표로 했고, 벡터 DB 와 LLM 재순위는 대상과 자료가 늘어날 때 적용을 검토한다.
- 질의 단계: LLM 이 질문을 여러 형태로 확장하고, 이를 부분 그래프 조회 템플릿으로 바꿔 Cypher 를 자동 생성한다. 그래프 조회 결과와 벡터 검색 결과를 합쳐 LLM 의 맥락으로 주고, 답변을 모아 하나의 응답을 만든다. 정해진 순서(chain)와 에이전트를 함께 쓴다. → 이번 시험에는 KG 조회 결과를 LLM 의 근거로 주는 원리를 적용하고 Cypher 는 CQ 마다 미리 정한 템플릿을 썼다. 질의 확장, Cypher 자동 생성, 벡터 검색, 에이전트는 답할 수 있는 질문의 범위를 넓힐 때 적용을 검토한다.
- 평가: RAGAS 지표와 전문가 정성 평가를 쓴다고 밝혔으며, 정식 평가는 향후 과제로 두었다.

## D.2 Huang 외(2026) — 저장

- 추출한 지식을 JSON 으로 통합하고, DDO 의 의미·계층 관계에 따라 Python 스크립트가 JSON 키-값을 읽어 Cypher 문을 만든다. 이 문들로 DDKG 의 Entity 를 일괄 생성하고 Entity 사이의 관계를 만들어 Neo4j 에 적재한다.
- 저장소로 Neo4j 를 고른 이유: Cypher 로 조회·추가·삭제·수정이 효율적이고, 다중 홉 추론과 복잡한 질의에 강하다.
- 구축한 DDKG 는 Entity 201,362 개, Relation 953,447 개이다. → JSON 대신 CSV 를 모은 형태로 쓴 것 외에는 같은 순서(모음 → 스크립트로 Cypher 생성 → 일괄 적재)를 따랐다.

## D.3 Zhou 외(2023) — KG 밖에 두는 저장소

- 무엇인가: 댐 안전 모니터링을 위한 BIM·온톨로지 기반 지식관리 저널 논문이다(Automation in Construction). OntoDSMS 는 이 논문이 만든 댐 안전 모니터링 시스템(DSMS) 도메인 온톨로지의 이름이고, D2RQ 는 이 논문이 쓴 기존 오픈소스 도구이다.

- 데이터를 원래 저장소에 둠: D2RQ 는 관계형 DB 에 접근하는 가상 RDF 그래프를 제공하며, RDB 의 내용을 RDF 저장소에 복제하지 않고 RDF 로 접근하게 한다(원문 6 쪽).
- 키로 대응: 매핑 파일에서 표의 ID 열 값으로 인스턴스 식별자를 만들고(uriPattern), 표는 Class, 열은 DatatypeProperty, 행은 인스턴스, 셀은 값에 대응시킨다(원문 6 쪽 그림 3, 7 쪽).
- 질의할 때 읽어 옴: D2RQ 는 RDF 질의를 SQL 문으로 번역해 RDB 에서 실행한다(원문 7 쪽). 이력 데이터와 실시간 수집 데이터를 MySQL 에 둔 채 온톨로지와 대응시켰고(원문 10 쪽), 질의 1 에서 4 번 구간 수직 변위 센서의 2015-2-28 이후 모니터링 데이터를 조회했다.
- → 데이터를 원래 저장소에 두고 키로 KG 와 이어 질의할 때 읽어 오는 원리를 따랐다. 매핑 파일과 SQL 번역 대신, KG 에서 키를 얻은 뒤 코드가 CSV 에서 행을 찾는다.

# 부록 E. 참고문헌


- Huang, J., Hou, W., Zhang, S., Du, Z., & Yin, J. (2026). Dam defects ontology and knowledge graph construction from multi-source hazard data using large language models. Advanced Engineering Informatics, 76, 104987.
- Purohit, S., Chin, G., Mackey, P. S., & Cottam, J. A. (2024). GraphAide: Advanced Graph-Assisted Query and Reasoning System. 2024 IEEE International Conference on Big Data. https://doi.org/10.1109/BigData62323.2024.10825705
- Zhou, Y., Bao, T., Shu, X., Li, Y., & Li, Y. (2023). BIM and ontology-based knowledge management for dam safety monitoring. Automation in Construction, 145, 104649.